

Chapitre 2 : Listes en OCaml

I Théorie et définition inductive

En programmation fonctionnelle, la liste est une structure de données fondamentale, homogène (tous les éléments ont le même type) et de taille dynamique. Contrairement aux tableaux, les listes ne sont pas indexées en mémoire de manière contiguë.

Définition : Une liste en OCaml est définie de manière **inductive** (ou récursive) :

- Soit la liste est vide, notée `[]`.
- Soit elle est composée d'un élément de tête x (de type `'a`) et d'une suite d'éléments appelée reste ou queue xs (de type `'a list`). On la note alors $x :: xs$.

L'opérateur `::` (prononcé *Cons*) permet de construire une liste en ajoutant un élément en tête. Il est associatif à droite.

Exemple : La liste contenant les entiers 1, 2 et 3 peut s'écrire explicitement `[1; 2; 3]` ou de manière équivalente `1 :: 2 :: 3 :: []`.

Attention Ne confondez pas le séparateur d'éléments dans une liste qui est le point-virgule (`;`) avec la virgule (`,`), qui sert à définir des n-uplets (tuples, nous reverrons cela dans le prochain chapitre).

II Analyse de complexité et représentation mémoire

En mémoire, une liste est une chaîne de pointeurs (une liste simplement chaînée). Chaque maillon contient une valeur et un pointeur vers le maillon suivant. Cette implémentation dicte directement la complexité des opérations de base sur les listes :

- **Ajout en tête (`::`) :** Cette opération se contente de créer un nouveau maillon pointant vers l'ancienne liste. C'est une opération en temps constant, soit en $\Theta(1)$.
- **Accès à la tête :** Immédiat, en $\Theta(1)$.
- **Accès au n -ième élément ou calcul de la taille :** Nécessite de parcourir la liste maillon par maillon. Pour une liste de taille N , cela s'effectue en $\Theta(N)$.

Remarque : Puisque les structures de données en programmation fonctionnelle pure sont immuables, l'ajout en tête ne copie pas la liste existante ; elle est partagée en mémoire (*structural sharing*), ce qui rend le `::` extrêmement efficace.

III Le module `List`

Vocabulaire : En OCaml, un **module** regroupe un ensemble de types et de fonctions liés à une même thématique. Le module `List` de la bibliothèque standard fournit les fonctions indispensables pour manipuler les listes sans avoir à les réécrire.

Pour appeler une fonction d'un module, on utilise la notation pointée : `Nom_du_module.nom_fonction`.

A Fonctions de base du module `List`

Voici les fonctions incontournables, leurs signatures, leurs rôles et leurs complexités.

1 `List.hd` et `List.tl`

- **Signatures** : `List.hd` : `'a list -> 'a` et `List.tl` : `'a list -> 'a list`
- **Rôle** : `hd` (*head*) renvoie le premier élément ; `tl` (*tail*) renvoie la liste privée de son premier élément.
- **Complexité** : $\Theta(1)$ pour les deux opérations.

✗ **Attention** ✗ Ces deux fonctions lèvent une exception `Failure` si la liste passée en argument est vide. Il est souvent préférable d'utiliser le pattern matching.

2 `List.length`

- **Signature** : `List.length` : `'a list -> int`
- **Rôle** : Renvoie le nombre d'éléments de la liste.
- **Complexité** : $\Theta(N)$ où N est la longueur de la liste.

3 `List.append` ou opérateur `@`

- **Signature** : `List.append` : `'a list -> 'a list -> 'a list`
- **Rôle** : Concatène deux listes pour n'en former qu'une seule.
- **Complexité** : $\Theta(N)$ où N est la taille de la **première** liste uniquement (car elle doit être parcourue et recrée pour pointer sur la deuxième).

IV Exemples de code et Pattern Matching (motifs)

La manière idiomatique de manipuler les listes est d'associer la récursivité au filtrage par motif (*pattern matching*).

A Exemple 1 : Calculer la somme d'une liste d'entiers

On applique un pattern matching sur la structure inductive de la liste.

```

1 let rec somme liste =
2   match liste with
3   | [] -> 0 (* Cas de base : la liste est vide *)
4   | tete :: reste -> tete + (somme reste) (* Cas récursif *)

```

Analyse de complexité : La fonction visite chaque maillon une seule fois. La complexité temporelle est en $\Theta(N)$.

B Exemple 2 : Implémentation alternative de la concaténation

Voici comment reconstruire l'opérateur `@` à la main :

```

1 let rec concatener l1 l2 =
2   match l1 with
3   | [] -> l2
4   | tete :: reste -> tete :: (concatener reste l2)

```

Analyse de complexité : On observe bien graphiquement que seul le premier argument `l1` est déstructuré élément par élément, justifiant sa complexité en $\Theta(\text{length}(l1))$.

V Fonctions d'ordre supérieur

Le module `List` brille par ses fonctions prenant d'autres fonctions en paramètres (fonctions d'ordre supérieur).

Contents

I	Théorie et définition inductive	1
II	Analyse de complexité et représentation mémoire	1
III	Le module <code>List</code>	1
A	Fonctions de base du module <code>List</code>	1
1	<code>List.hd</code> et <code>List.tl</code>	2
2	<code>List.length</code>	2
3	<code>List.append</code> ou opérateur <code>@</code>	2
IV	Exemples de code et Pattern Matching (motifs)	2
A	Exemple 1 : Calculer la somme d'une liste d'entiers	2
B	Exemple 2 : Implémentation alternative de la concaténation	2
V	Fonctions d'ordre supérieur	2
1	<code>List.map</code>	3
2	<code>List.filter</code>	3